

New York University — Tandon School of Engineering
Department of Technology Management and Innovation
IE-GY 9113B — Systems Integration: From ERP to Agentic AI
Study Note — Week 0: The ERP Floor (Optional Pre-Course Primer)
Module 1: The Integration Foundation

Readings for this week	
Foundational reading	Thomas H. Davenport, “Putting the Enterprise into the Enterprise System,” Harvard Business Review, Vol. 76, No. 4 (July–August 1998), pp. 121–131. Available through NYU Libraries (Gale Academic OneFile)
Current-state reading	McKinsey, “Bridging the great AI agent and ERP divide to unlock value at scale” (January 2026). Read it open-access here.
Supporting video	Michaela Goss, “An explanation of SAP ERP, ECC and S/4HANA,” Informa TechTarget / Eye on Tech. Watch the video.
Optional	SAP, “What is ERP: The Essential Guide” — vendor reference, for orientation only.

An Optional Head Start

This primer is optional — a head start for anyone who wants one, not a requirement. Read it whenever suits you: before the term starts, during the first week, or not at all. Nothing here is graded, and no one is behind or penalized for skipping it. Everything essential gets taught again, from the ground up, in Week 1.

What follows is deliberately light: what an ERP or integration backbone actually is, how ERP types differ, and just enough of a BPMN / process-mapping vocabulary that the terms won't be new when they matter. It sets a tone more than it sets a test — a first look at the floor this course is built on, at whatever pace you'd like to take it.

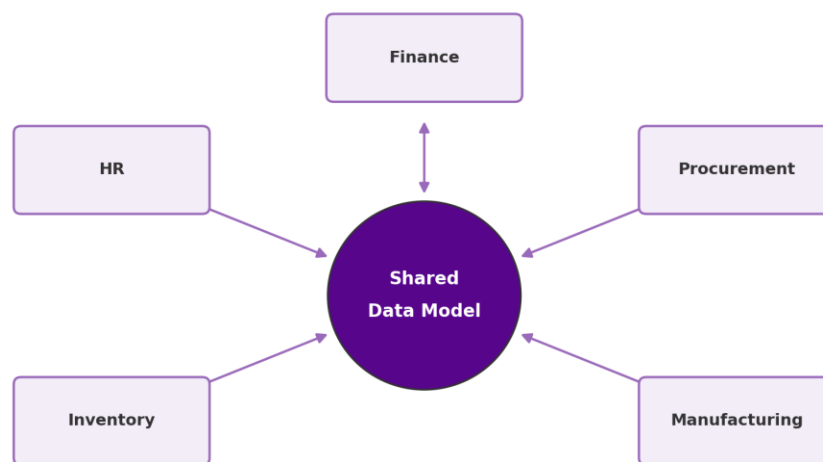
Whatever company you eventually choose to work with — a global CPG manufacturer running its planning off SAP, a mid-market manufacturer on NetSuite, a Series B startup stitching together Stripe, Shopify, and QuickBooks, or a small services firm running on HubSpot and Notion — the question this primer asks is the same question Week 1 asks at every one of those scales: what does it actually mean for a company's systems to work together, and why is that so much harder than it sounds?

A Company Doesn't Have One Brain — Until You Build It One

Every company generates far more data than any single person could hold in their head: purchase orders, invoices, inventory counts, payroll runs, shipping schedules, customer records. The uncomfortable fact about most large organizations is that this information is not kept in one place. It is scattered across dozens, sometimes hundreds, of separate systems — one for finance, another for inventory, another for HR — each one built to serve a single function and blind to what the others know.

That fragmentation is not merely an IT inconvenience; it shows up directly in the business. When a sales system cannot talk to a production-scheduling system, the factory floor and the promises made to customers start to drift apart. When sales and marketing data does not reconcile with financial reporting, leadership ends up making decisions on instinct rather than evidence. Put bluntly: a company with fragmented systems is, in a very real sense, a fragmented company — its different parts literally cannot see each other.

An enterprise system — what most people mean when they say “ERP” — is the attempt to fix this with one shared database that every function reads from and writes to. Enter a customer order once, and that single event should ripple automatically into inventory, into production scheduling, into the general ledger, into a sales rep's commission — without anyone re-keying the same fact into three different screens. That is the entire promise, and it is a genuinely powerful one when it works.



One entry updates every connected function in real time — this is what “integration” means in practice.

Figure 1. The shared data model: one comprehensive database feeding and drawing from every business function. Adapted from the enterprise-system concept in Davenport (1998).

Zoom out on what that database actually has to hold, and you can see why the promise is so ambitious. A working enterprise system typically spans four broad zones of the business: financial data (the ledgers, receivables and payables, cost accounting); human-capital data (payroll, time, personnel records); operations and logistics data (inventory, materials planning, production, purchasing, shipping); and commercial data (orders, pricing, sales planning). None of these zones is optional if the promise of “one shared brain” is going to hold — leave one out, and the company still has a blind spot exactly where that zone used to be.

Integration means one entry, many updates.

A technology backbone earns the label “integrated” the moment a single transaction — a customer order, a purchase requisition, an invoice — automatically updates every function that depends on it, without a human re-entering the same fact twice. Everything else this course teaches is built on top of that one mechanic.

One Shared Model, Several Different Shapes

So far, this section has described what any enterprise system does. It is worth being just as clear about how they differ — because the ERP running a Fortune 500 CPG manufacturer's global operations looks nothing like the ERP running a 50-person services firm, and the difference is not only company size, which the four course archetypes already capture. Systems also differ along two lines that cut across size entirely.

The way the market usually sorts these systems is a practical, industry vocabulary rather than a formal academic classification — it is how analysts and buyers segment the landscape, and it is worth knowing in those terms. Three dimensions do most of the work:

Dimension	What varies	Why it matters
Deployment	On-premise (runs on the company's own servers), cloud (hosted by the vendor), or hybrid (a mix of both)	On-premise gives control and deep customization; cloud lowers IT overhead and speeds up rollout
Scope	Generalist / horizontal (one configurable system built to serve many industries) versus industry-specific / vertical (built around one sector's workflows and compliance needs)	A generalist system treats finance and HR the same way for any company; a vertical system bakes in things like process-manufacturing formulas or healthcare compliance from the start
Market tier	Tier I (large global enterprise, broadly enterprises above ~\$750M in revenue), Tier II (mid-market), or Tier III (small business)	A different lens from the four company archetypes you'll use all semester — the two often line up, but not always

These three lines cut across each other rather than replacing the archetype lens you will use all semester. The same system can be a cloud, Tier II, generalist platform for one company's finance function while a different, industry-specific module handles its manufacturing floor. What matters for now is simply this: “ERP” is not one thing. Part of reading a company's backbone correctly, starting in Week 1, is placing it on these lines — not only on the size scale.

The Software Rarely Breaks. The Business Around It Does.

The historical record on enterprise systems is a useful cold shower before you get excited about what they promise. Some well-known projects have gone badly wrong for reasons that had little to do with the underlying code: one distributor has pointed to its system as a contributor to its own bankruptcy; a major oil company's regional unit spent hundreds of millions of dollars on an implementation it later abandoned when a merger partner objected; a personal-computer maker found that a rigid, centralized system did not fit the decentralized way it had decided to run the business; another manufacturer found itself overwhelmed by the sheer scale of organizational change the project demanded; and one chemical company spent seven years and roughly half a billion dollars on a mainframe-based system, only to decide to start over on a newer architecture.

None of these are morality tales about bad software. They are stories about companies that treated a business-transformation decision as if it were a technology-purchasing decision — and paid for the mismatch.

Set against that record, one case is worth walking through because it shows the opposite pattern. A mid-sized North American chemicals subsidiary, formed through a series of mergers, found itself with twelve business units that could not coordinate: ordering systems disconnected from production, sales forecasts disconnected from budgets, each unit reporting its own financials independently. Placing and paying for a single order could mean multiple phone calls and a stack of separate invoices. Internally, a routine order took four days and seven handoffs between departments to process — despite representing only about four hours of actual work.

The company chose the same kind of enterprise system many of its competitors were also installing. What made the outcome different was what leadership decided to focus on. Rather than treating the rollout as a technology initiative, they used it as an occasion to redesign the organization itself — concentrating the effort on four processes that most affected customers directly: materials management, production planning, order management, and financial reporting. Accounts-receivable and credit functions that had been scattered across units were merged into one, so a customer's orders could be billed on a single invoice instead of several. Customer service was consolidated into one team with one point of contact. A new role — a demand manager, responsible for connecting sales forecasts to real-time production capacity — was created because the information needed to do that job had never existed in one place before.

The results were not just technical. Customer confirmation times improved from an average of roughly five calls per order toward a target of one. Inventory write-offs and unplanned production stoppages fell. Nine of the twelve business units were live on the new system ahead of schedule and under budget. The company's own account of the effort is direct about why: its leaders “stressed the enterprise, not the system.”

The system reflects the organization. It rarely fixes it for free.

An enterprise system will faithfully encode whatever decisions a company makes about its own processes — good or bad. Installed on top of an unreconciled organization, it will make the fragmentation faster and more expensive. Installed alongside a genuine redesign of who does what and how, it becomes the backbone this course is named for.

Why an Argument from 1998 Still Opens This Course

It would be reasonable to ask why a course about agentic AI opens with an argument written before most of you were born. The answer is that the same tension — whether a company treats its shared data and processes as the main event or as an afterthought — has resurfaced almost exactly, with AI agents standing in the role software once played.

Recent industry research on the relationship between AI investment and enterprise systems describes a widening gap: as budgets shift toward generative AI and autonomous agents, spending on the core infrastructure and architecture that those agents actually depend on — the ERP layer — has been dropping. Only a minority of companies report a measurable, enterprise-level profit impact from their AI

initiatives so far. Enthusiasm for agents is real; the underlying data and process foundation those agents need to act on reliably is, in a lot of organizations, being treated as legacy baggage rather than as the thing that makes any of it scalable, auditable, or safe.

That is precisely backwards. An AI agent does not conjure the state of a company's inventory, its open orders, or its cash position out of nowhere — it senses and acts through the same shared data model this note has just described. An agent layered on top of a fragmented, unreconciled backbone will not fix the fragmentation; if anything, it will act on bad data faster and with more confidence than a human would have. The floor still has to be solid before anything gets to stand on it, autonomous or not.

Learning to Read a Process Before You Map One

Before you can diagnose why a backbone is or isn't creating value — the work of Weeks 1 and 2 — and before you map one yourself in Lab 1, it helps to have a small, shared vocabulary for talking about a business process. Business Process Model and Notation (BPMN) is the standard visual language for this, and only a handful of its building blocks are needed to get started.

Element	What it represents	The question it answers
Event (start / end)	A point where a process begins or concludes	What triggers this, and what marks it done?
Task / activity	A discrete unit of work performed by a person or system	What actually happens at this step?
Gateway	A decision point — where the routing choice is made (approve or reject, route one way or another)	What gets routed where, and by what rule?
Sequence flow	The arrow connecting one step to the next	What has to finish before this can start?
Swimlane	A lane assigned to a role, system, or department	Whose job is this step?

That gateway symbol is worth remembering by name: Week 1 and Lab 1 both call it a decision point, and you will eventually label these on a live process map. Hold the rest of this vocabulary lightly for now — Lab 1 asks you to map a real end-to-end process using exactly these elements, marking both the control points — where the business commits or data is created — and the decision points — where the routing choice is made — concepts you will meet properly in Week 1. This primer's only job is to make sure the words aren't new when they matter.

What This Primer Hands to Week 1

If you've read this far, you now have a first look at the floor: what an enterprise system actually is and the shared data model underneath it, a sense of how ERPs differ beyond scale alone, why these projects tend to succeed or fail on organizational grounds more than technical ones, why an argument from 1998 still matters for a course about AI agents, and a small process-mapping vocabulary that already includes the decision point. Week 1 builds its central argument directly on top of this territory —

why a backbone functions as a company's single source of truth regardless of its size, and why a backbone can work exactly as designed and still fail to create the value it promised — and it does so from first principles, whether or not you've read a word of this primer.

Team formation, tool setup, and your first look at the capstone storyboard all happen once Week 1 begins — there's nothing to prepare on any of that front here.

Running Glossary

Integration backbone / ERP: the shared technology and data layer — whether a single ERP package or a connected multi-SaaS stack — that a company relies on to run its core operations.

Shared data model: a single database structure that multiple business functions read from and write to, so that one transaction updates every function that depends on it.

ERP types: systems vary by deployment (on-premise, cloud, hybrid), scope (generalist / horizontal versus industry-specific / vertical), and market tier (Tier I, II, III) — market vocabulary distinct from the four course archetypes' scale axis.

Single source of truth: previewed here, defined fully in Week 1 — the idea that one authoritative version of a piece of data exists, rather than several disconnected copies.

BPMN (Business Process Model and Notation): a standard visual language for mapping business processes using events, tasks, gateways, flows, and swimlanes.

Control point: where the business commits or data is created (previewed here, full treatment Week 1).

Decision point: where the routing choice is made — approve or reject, route one way or another (previewed here via the BPMN gateway, full treatment Week 1 and Lab 1).